When Is an RL Problem a Diffusion Problem?
A Ten-Domain Empirical Map

Aamer Khani*

August 2026

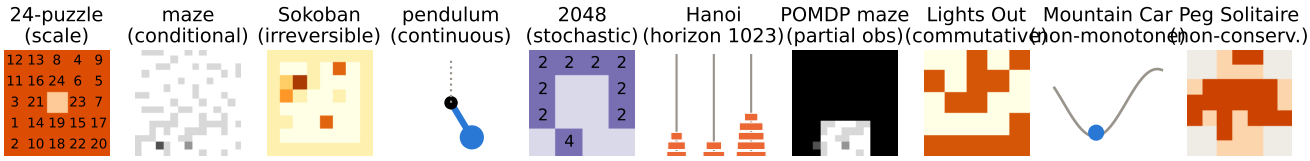


Figure 1: The ten domains, each chosen to remove one property that the Rubik's Cube—where scramble inversion excels [8]—quietly provides. Every panel is a real environment instance from the released code.

Abstract

Scramble inversion—train a denoiser to predict the action that undoes the last step of a random walk from the goal, then act by reverse-process rollout—beats deep approximate value iteration (DAVI) on the Rubik's Cube at a fraction of the compute [8]. Does "every RL problem can be a diffusion problem"? We answer with a ten-domain *assumption ladder*, each domain deleting one property of the cube: group structure (24-puzzle), fixed goal and environment (procedural mazes), invertible actions (Sokoban), discreteness (pendulum), determinism (2048), bounded horizon (Tower of Hanoi, optimal solutions of 1023 moves), full observability (POMDP mazes), ordered solutions (Lights Out), monotone progress (Mountain Car), and conservation (Peg Solitaire). Every domain is run under a matched-architecture, matched-compute protocol against a value baseline on one consumer GPU, with exact oracles wherever the state space permits and mechanical replay verification of every claimed solution. The resulting map is sharp and mechanistic. Scramble inversion *dominates at scale*: 100.0% vs. 0.6% on the 24-puzzle (7.7×10^{24} states) at matched iterations. It survives conditioning, partial observability, commutativity, and non-monotone dynamics. It has two repairable failure modes: with irreversible actions its reverse-only data is deadlock-blind (Sokoban: $65.2\%(64.9 - -65.5)$ vs. DAVI's $82.7\%(81.2 - -83.7)$), which a zero-training hybrid—denoiser proposes, value function vetoes—not only repairs but turns into the best solver (93.0%); and with continuous actions, naive ϵ-regression collapses to a useless posterior mean ($0.0\%(0.0 - -0.0)$ swing-up) while the same data with a distributional head reaches $100.0\%(100.0 - -100.0)$. Extreme horizon (Hanoi) exposes a third, diffusion-native failure shared by both methods: non-backtracking noising walks never mix across the 1023-move diameter (1200-step walks stay within distance 125), and each method is near-perfect inside the covered shells and zero beyond—the schedule, not the learner, is the ceiling. Finally there is one hard boundary: stochastic dynamics (2048), where the deterministic reverse process describes a game that is never played and the denoiser performs at random-play level—and a spawn-aware noising walk that matches the state marginal does not help, because the undo-labels themselves answer the wrong question. We conclude with a practitioner's checklist and release all ten environments, trainers, and logs.

*Experiments, implementation, and manuscript preparation were carried out with the assistance of Claude (Anthropic). Correspondence: `aamer.khani.1@gmail.com`.

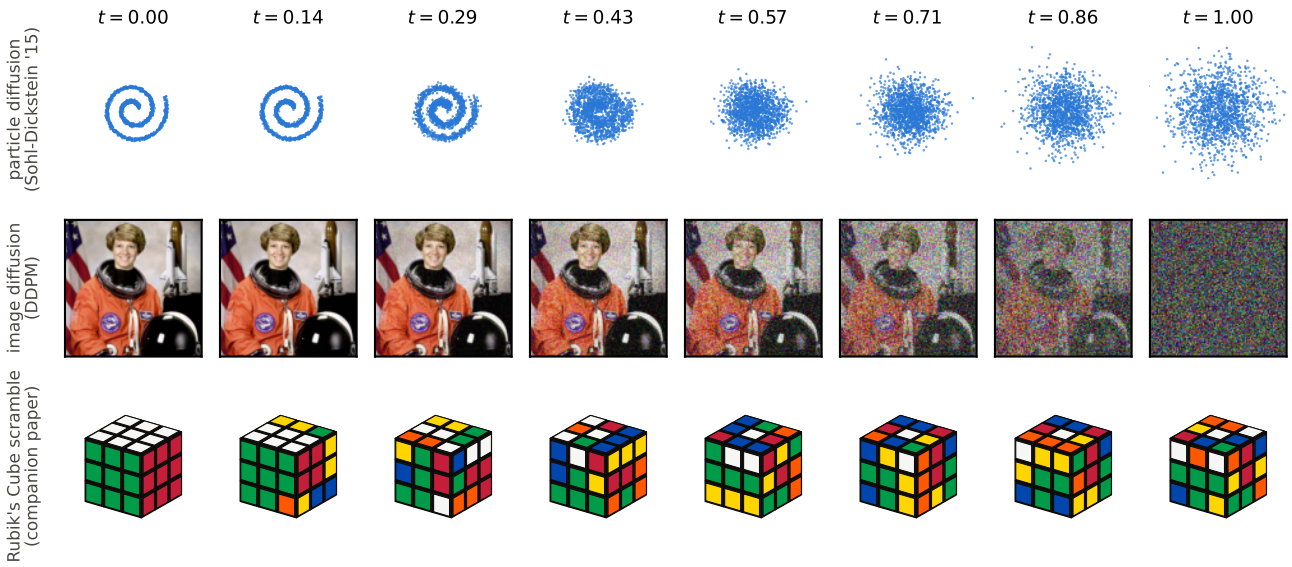


Figure 2: The reference processes, kept separate from the ten study domains. One shared schedule t=0→1: particle diffusion off a spiral (top, 10), image diffusion (middle, 6), and the Rubik's Cube noising walk of the companion paper [8] (bottom; real engine states, scramble depth = $\lfloor 30t \rceil$). Every domain figure in this paper is this same picture with the noise operator swapped.

1 Introduction

The companion study [8], which shares an author with the present paper, formalized cube solving as discrete denoising diffusion on a Cayley graph: random moves are the noise, scramble depth is the timestep, and a categorical denoiser trained to invert the last scramble move solves fully scrambled cubes by reverse-process rollout, matching DAVI [1] at 2.8× less wall-clock. Figure 2 places that formulation in its lineage: the same t=0→1 schedule that diffuses a particle cloud off a spiral [10] and dissolves a photograph under Gaussian noise [6] scrambles a cube under random moves—three noise operators, one recipe.

That result invites the strong hypothesis, posed by the first author: *every reinforcement-learning problem can be recast as diffusion, and may train faster*. This paper maps the hypothesis's actual extent.

The cube bundles many conveniences at once: a group action, one fixed goal, invertible moves, discrete states and actions, deterministic dynamics, modest solution lengths, full observability, ordered solutions, monotone usefulness of progress, and conservation of the state's "material". Any of these could be the load-bearing one. We therefore built ten environments (Figure 1), each deleting exactly one property, and ran the identical recipe against a matched value-learning baseline in every one. Three principles carried over from the companion paper: (i) *matched protocol*—same residual-MLP backbone, batch, optimizer, and schedule, so the training objective is the only variable; (ii) *exact validation*—five of the ten domains admit exhaustive oracles (8-puzzle: 181,440 states; Hanoi: 59,049; Lights Out: GF(2) algebra; per-instance BFS fields for both maze variants), and every environment ships a test that reproduces a known exact quantity before any training is trusted; (iii) *replay verification*—no solver output is believed until its action sequence is re-executed in the environment.

#	Domain	Removes	Outcome (greedy, matched)	Verdict
1	24-puzzle	group structure	**denoiser** 100.0% vs 0.6%	✓
2	mazes	fixed goal/env	tie: 78.0%(76.0 – −82.0) vs 79.2%(76.6 – −81.5)	✓
3	Sokoban	invertibility	value wins (65.2%(64.9 – −65.5) vs 82.7%(81.2 – −83.7)); **hybrid** 93.0%	△
4	pendulum	discreteness	0.0%(0.0 – −0.0) → 100.0%(100.0 – −100.0) with distributional head	△
5	2048	determinism	denoiser = random (7.1%(7.1 – −7.2) vs 9.0%)	✗
6	Hanoi	bounded horizon	both ∼2% uniform; 100% inside covered shell	✗
7	POMDP maze	full observability	84.6%(83.3 – −85.3) vs 86.2%(85.9 – −86.9)	✓
8	Lights Out	ordered solutions	100.0%(100.0 – −100.0) vs 33.9%(33.8 – −34.0)	✓
9	Mountain Car	monotone progress	95.8%(92.2 – −99.0) vs 41.8%(0.0 – −66.0)	✓
10	Peg Solitaire	conservation	99.7%(99.5 – −99.8) vs 0.0%(0.0 – −0.0)	△

Table 1: The assumption ladder. ✓ = scramble inversion is doable and competitive or dominant; △ = doable with a specific, identified adaptation (hybrid value veto; distributional head); ✗ = fails for a structural reason. Throughout the paper, $a\%\,(b\text{–}c)$ denotes the mean over three independent training seeds with min–max range; single percentages are single-seed or exhaustive evaluations.

2 Background: scramble inversion on the Rubik's Cube

This paper is self-contained; this section compresses everything a reader needs from the companion cube study [8].

Formulation. The cube's configuration is a discrete state s; the twelve quarter-turns are actions on it. Image diffusion corrupts a photo with Gaussian increments and learns the reverse conditional; scramble inversion corrupts the *solved* cube with a walk of uniformly random moves and learns the reverse conditional there: given a state produced by a k-move scramble, predict the inverse of the move applied last. Scramble depth plays the role of the timestep t (drawn uniformly, a linear schedule), and the categorical cross-entropy on the undo-move is the exact discrete analogue of ϵ-prediction: both regress the noise increment that produced the current state (Figure 2). The objective itself appears outside the diffusion frame in Takano [12]; the companion paper's contribution is the diffusion formulation, its schedule analysis, and a matched-compute comparison. Solving is reverse-process rollout: repeatedly apply the predicted undo-move until the goal appears (optionally widened into a beam search).

Results carried into this paper. On the 2×2×2 cube (3,674,160 states, exhaustively enumerable), the trained denoiser solves 100% of *all* states—checked against an exact breadth-first-search oracle, with greedy rollout alone at 99.989%. On the 3×3×3 (4.3×10^{19} states), it solves 1000/1000 cubes scrambled with 100 random moves. The efficiency gap is the motivation for everything that follows: the denoiser trains at 28.7 iterations/s versus DAVI's 5.8 on identical hardware—each DAVI update must expand and evaluate all twelve successors of every sample to form its bootstrap target $1+\min_a V(s')$, while the denoiser needs one forward pass on a directly supervised label—reaching its solve rate in 4.9 h versus 13.9 h (2.8×), and dominating at every beam width tested. DAVI keeps one edge: shorter solutions (23.0 vs. 29.9 moves on average), the classic trade of solution quality for training cost. Ablations showed the recipe is robust on the cube: schedule shape barely matters ($\geq$ 99.96% across all tested), and timestep conditioning is optional. The cube, however, is a Cayley graph of a group—homogeneous, invertible, deterministic, fully observed—and every one of those properties is an assumption the rest of this paper deletes.

3 The recipe, its baseline, and one architecture

Recipe. Sample a noising walk from the goal using a reverse transition kernel (the inverse moves themselves when actions are invertible; pulls, un-jumps, un-slides, or reverse-time integration otherwise), with depth

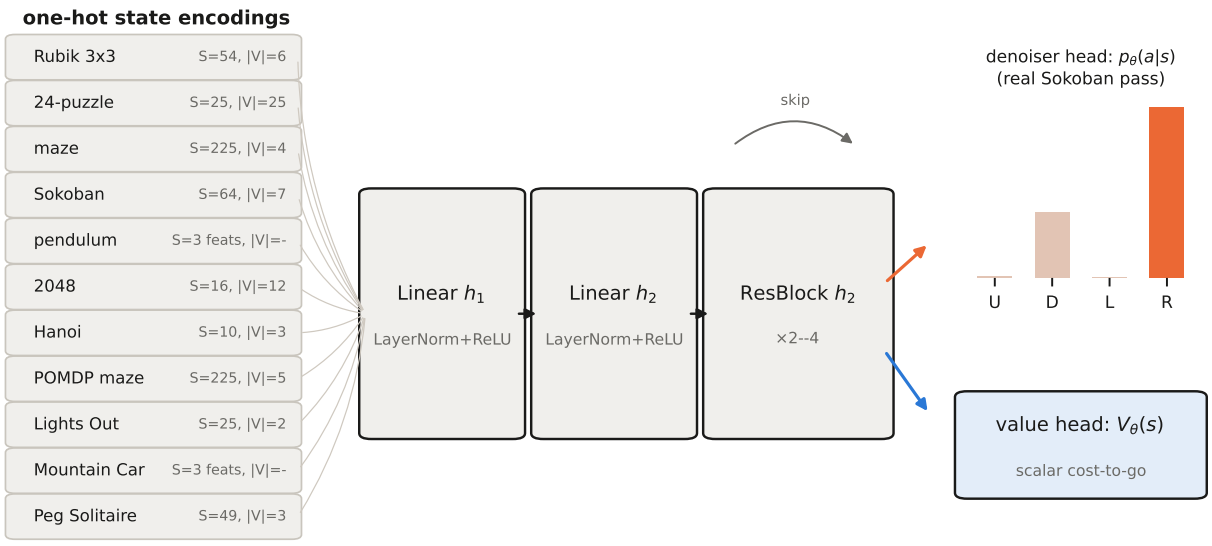
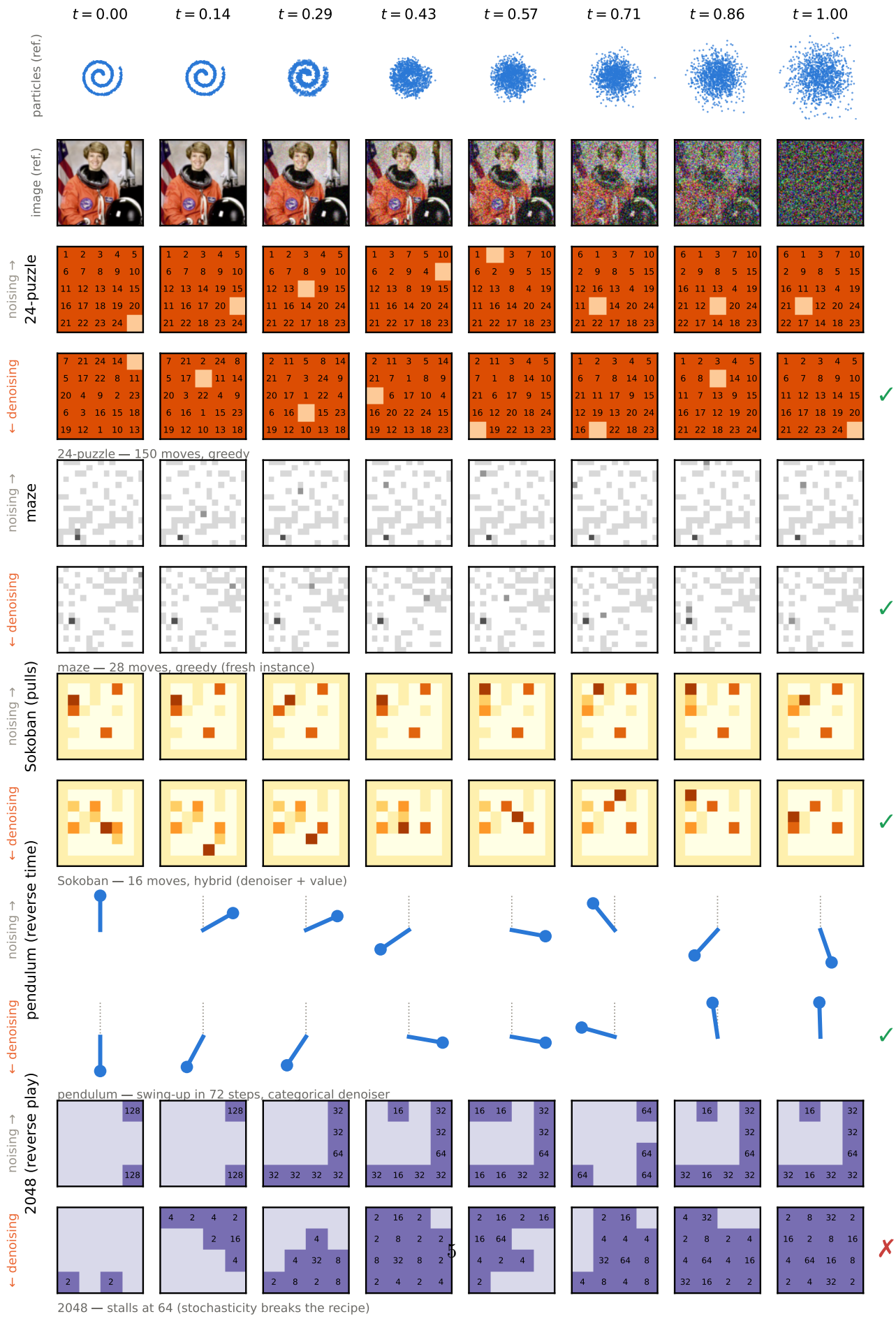


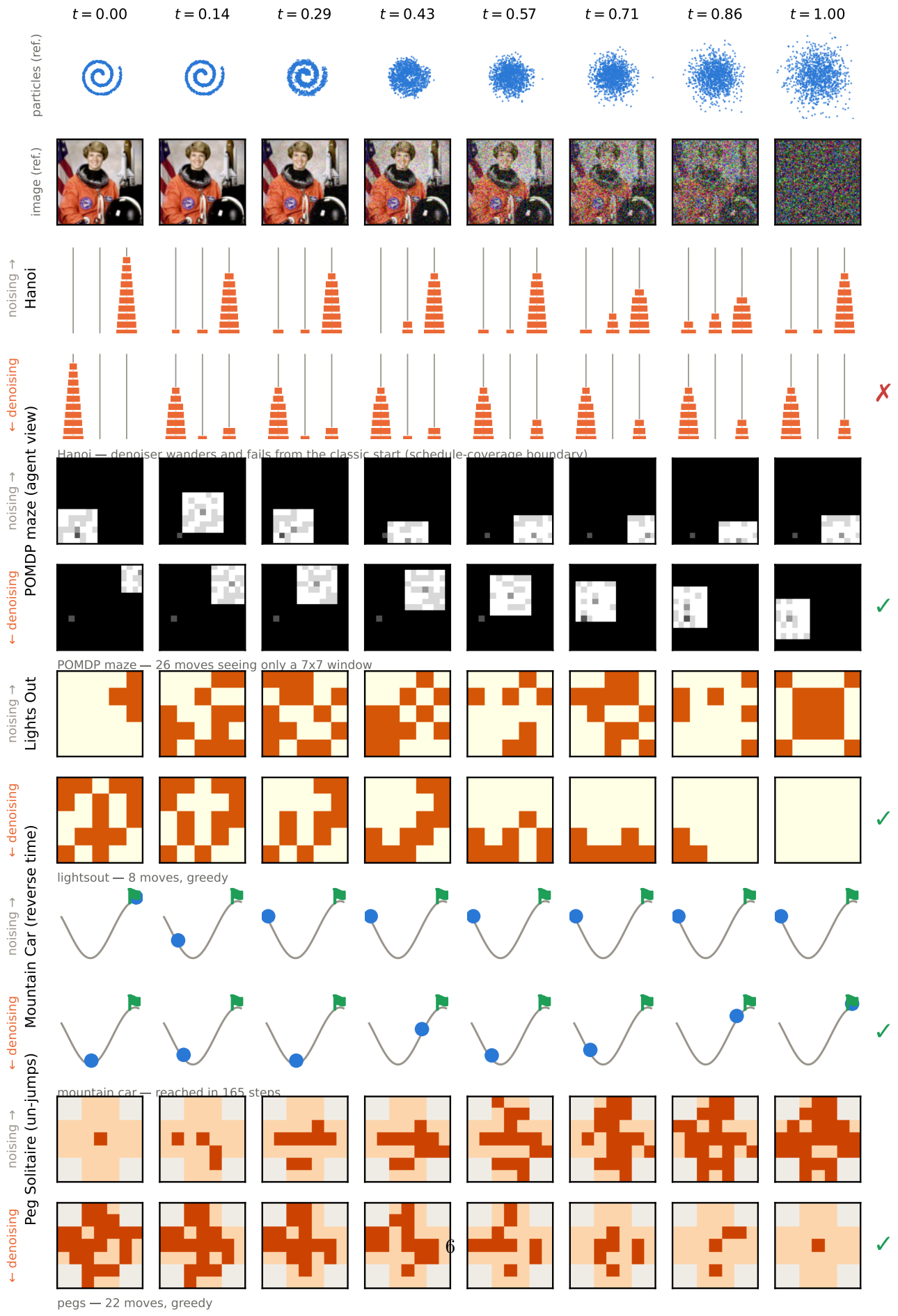
Figure 3: One architecture across the entire study. Left: per-domain one-hot encodings (S tokens, vocabulary $|V|$; the two continuous domains use 3 physical features instead). Middle: the shared LayerNorm residual MLP. Right: the two heads; the denoiser head shows a *real* forward pass of the trained Sokoban policy.

$t \sim \text{Unif}\{1..K\}$; train $p_\theta(a \mid s)$ by cross-entropy to predict the forward action that undoes the last noise op; act by greedy rollout (beam search optional). Throughout, *shell coverage* denotes the training distribution the walk induces over *true* distances-to-goal: because walks backtrack, deep distance shells are systematically under-visited relative to their share of the state space, and the learned policy can only be as good as the shells its data actually reaches (quantified exactly in the companion paper; taken to its extreme by Hanoi below). **Baseline.** DAVI: a cost-to-go net trained by one-step bootstrapped targets over the *forward* dynamics, acting by 1-step lookahead—identical backbone, differing only in head and loss. **Architecture** (Figure 3): all ten domains and both objectives share one residual MLP; only the one-hot input width, the head, and the loss change.

4 Watching noising and denoising in every domain

Figures 4 and 5 are the study in pictures. To anchor the unfamiliar in the familiar, the top two rows of each figure show the *same* t=0→1 schedule applied to the two classic diffusion settings every reader already knows: a particle cloud diffusing off a spiral [10] and a photograph dissolving under Gaussian noise [6]. Every domain row below is that same picture with the Gaussian replaced by the domain's noise operator. For each domain the top strip runs the *forward schedule*: the goal state at t=0 progressively destroyed by the domain's noise operator—random slides, reverse-time torques, pulls, un-jumps, presses—through every intermediate state to full noise at t=1. The bottom strip runs the *trained reverse process* on a fresh fully-noised instance: each panel is an actual intermediate state of the greedy rollout, ending in the goal (✓) or not (✗). The 2048 row makes the stochasticity failure visible: forward play under random spawns has no relation to the deterministic reverse process the denoiser was trained on, and the board fills with unmergeable debris.


$t = 0.00$
$t = 0.14$
$t = 0.29$
$t = 0.43$
$t = 0.57$
$t = 0.71$
$t = 0.86$
$t = 1.00$
particles (ref.)
image (ref.)
noising →
24-puzzle
← denoising
24-puzzle — 150 moves, greedy
noising →
maze
← denoising
maze — 28 moves, greedy (fresh instance)
noising →
Sokoban (pulls)
← denoising
Sokoban — 16 moves, hybrid (denoiser + value)
noising →
pendulum (reverse time)
← denoising
pendulum — swing-up in 72 steps, categorical denoiser
noising →
2048 (reverse play)
← denoising
2048 — stalls at 64 (stochasticity breaks the recipe)


$t = 0.00$
$t = 0.14$
$t = 0.29$
$t = 0.43$
$t = 0.57$
$t = 0.71$
$t = 0.86$
$t = 1.00$
particles (ref.)
image (ref.)
noising →
Hanoi
← denoising
Hanoi — denoiser wanders and fails from the classic start (schedule-coverage boundary)
POMDP maze (agent view)
POMDP maze — 26 moves seeing only a 7x7 window
Lights Out
lightsout — 8 moves, greedy
Mountain Car (reverse time)
mountain car — reached in 165 steps
Peg Solitaire (un-jumps)
pegs — 22 moves, greedy

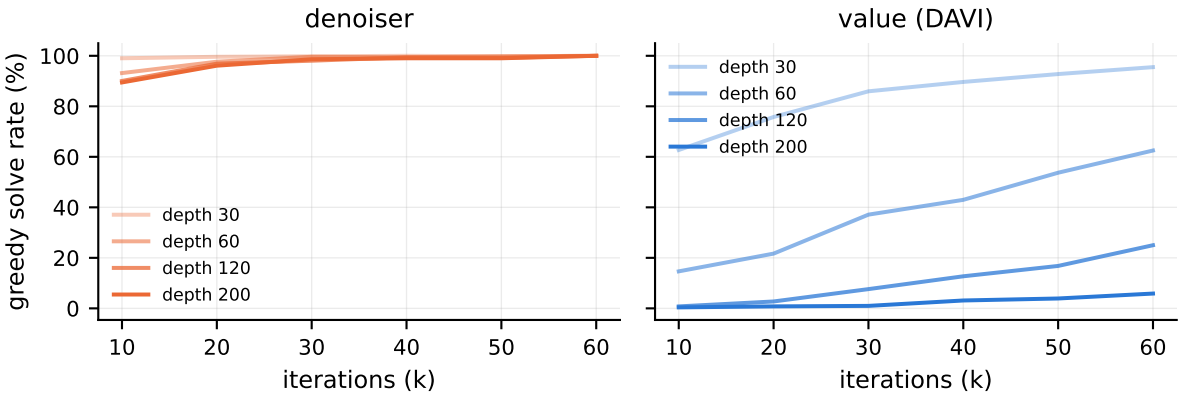


Figure 6: 24-puzzle greedy solve rate during training, by scramble depth (left: denoiser; right: DAVI, same axes and budget).

5 Results along the ladder

5.1 Scale is the denoiser's regime (24-puzzle, 8-puzzle, Hanoi)

On the 24-puzzle the denoiser reaches 100% greedy on every probe depth by 60k iterations and solves 500/500 thousand-move scrambles (99.8% with no search); DAVI at matched iterations solves 0.6% (Figure 6): bootstrapped values simply never propagate 100+ moves out in a 7.7×10^{24}-state space. The control puzzles calibrate the claim: on the exhaustible 8-puzzle both methods solve 100% of all 181,440 states (3-seed greedy: denoiser 100.0% (100.0–100.0), DAVI 100.0% (100.0–100.0)) and DAVI is nearly move-optimal (99.5% of its moves reduce true distance vs. 88.7%)—in small spaces value learning is the sharper tool. Hanoi delivers the study's sharpest lesson about the *schedule*. Walk-based probes report $\geq$99% for both methods—but the exhaustive sweep over all 59,049 states reports $\sim$2% for both (2.0% (2.0–2.1) / 1.7% (1.4–2.0)). The discrepancy is the noising process itself: non-backtracking walks do not mix on Hanoi's Sierpinski-structured graph, and 1200-step walks reach states of mean true distance just 36 (never beyond 125) of the 1023-move diameter. Stratifying the sweep by exact distance resolves the picture: the denoiser solves 100% of states within distance 50 and 96% within 51–100—precisely the covered shells—and 0% beyond 200. Neither horizon nor the objective is the boundary; *schedule coverage* is, taken to the extreme predicted by the shell-coverage analysis of the companion paper. We also tested the natural repair: *mixing* (backtrack-allowed) walks of length 1200, 5000, and 20,000 reach mean true distance only 22, 42, and 77 (max 79, 137, 278)—coverage grows diffusively, roughly as $\sqrt{K}$, against a diameter of 1023, so no feasible walk depth covers this geometry. The boundary is structural for walk-based noising; a fix must replace the forward process itself (explicit deep-state seeding, or noising kernels with super-diffusive mixing), not tune it.

5.2 Conditioning and partial observability are not boundaries

Procedural mazes (fresh walls and goal each instance; exact per-instance BFS oracles) end in a statistical tie: 78.0% (76.0–82.0) vs. 79.2% (76.6–81.5) on 20k held-out instances, with DAVI's paths closer to optimal (1.05× vs 1.21×) and the denoiser 4× faster to evaluate. Restricting the policy to a 7×7 window (POMDP) degrades both methods together (84.6% (83.3–85.3) vs 86.2% (85.9–86.9)): the information bottleneck, not the objective, is binding. Conditional denoising works as conditional diffusion suggests it should.

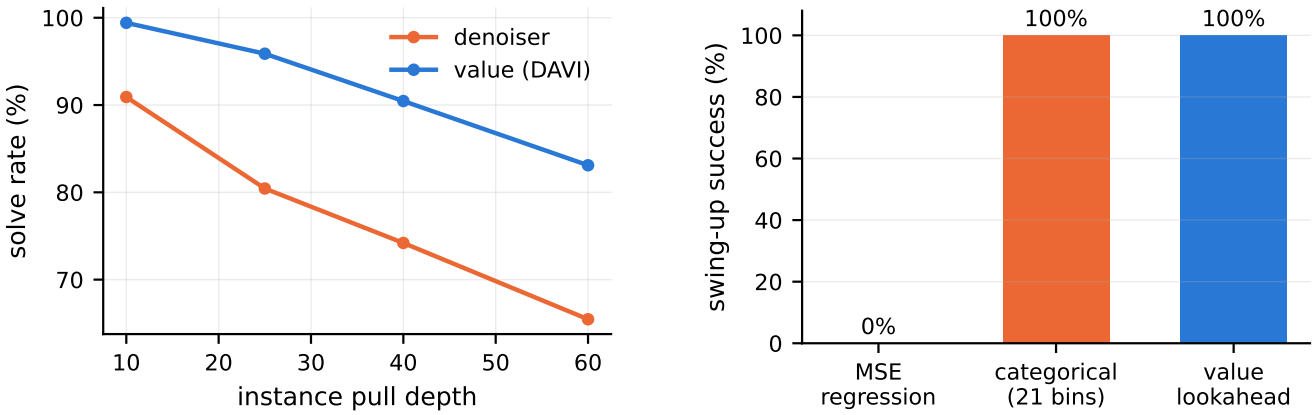


Figure 7: **Left:** Sokoban solve rate vs. difficulty; reverse-only training is deadlock-blind. **Right:** the pendulum ablation — ϵ-regression collapses to the bimodal posterior's mean; the same data with a 21-bin categorical head restores 100%.

5.3 Irreversibility: the failure with a constructive fix (Sokoban, Peg Solitaire)

Reverse (pull-based) walks cannot enter deadlocks, so the denoiser's data contains no signal about fatal pushes: it loses to DAVI at every depth ($65.2\%(64.9 - -65.5)$ vs $82.7\%(81.2 - -83.7)$ at pull-depth 60, Figure 7). The fix requires *no new training*: act by $\arg\max_a[\log p_\theta(a \mid s) - \lambda V(s'_a)]$—the denoiser proposes a direction, the value net (which bootstraps through forward dynamics and therefore knows dead states) vetoes it. With λ chosen on a held-out split, the hybrid solves 93.0% at depth 60, *surpassing both parents*. The choice is robust: validation solve rate is monotone in λ over $[0.1, 2]$ (79.4%, 86.8%, 94.5%, 95.5%, 97.2% at $\lambda = 0.1, 0.25, 0.5, 1, 2$), so any $\lambda \gtrsim 0.5$ recovers most of the benefit. Peg Solitaire, whose jumps destroy a peg each move (nothing is conserved and deadlocks abound), replays the pattern at rung ten: denoiser alone 99.7% (99.5–99.8), DAVI 0.0% (0.0–0.0) (depth-25 instances).

5.4 Continuity and non-monotonicity (pendulum, Mountain Car)

Naive *ϵ-regression* on the undoing torque fails absolutely ($0.0\%(0.0 - -0.0)$ swing-up): near hanging, reverse walks spin both ways, the posterior over undoing torques is bimodal, and its MSE-optimal mean is zero torque. The identical data and backbone with a 21-bin categorical head reaches $100.0\%(100.0 - -100.0)$ (value baseline: $100.0\%(100.0 - -100.0)$). The requirement is distributional output—exactly why diffusion models sample rather than regress—not discreteness. Mountain Car then confirms that *non-monotone* progress is no obstacle: reverse-time noising naturally demonstrates the momentum-building detour, and the categorical denoiser reaches the flag in 95.8% (92.2–99.0) of episodes (value baseline 41.8% (0.0–66.0)).

5.5 Commutativity (Lights Out)

When solutions are unordered sets, "the move that undoes the last noise op" is maximally ambiguous—any member of a minimal solution set is correct. The Bayes-optimal denoiser simply spreads mass over that set, and greedy rollout peels it off one press at a time: 100.0% (100.0–100.0) solve rate (DAVI 33.9% (33.8–34.0)), with solution lengths checked against the exact GF(2) optimum. Label ambiguity is averaged over, not compounded.

5.6 Stochasticity: the hard boundary (2048)

Trained on deterministic reverse play, the denoiser scores at random-play level in the real game (7.1%(7.1−−7.2) vs. random's 9.0% reaching the 256 tile), while a one-line greedy-merge heuristic reaches 61.8%. The mechanism is structural: with exogenous randomness, the reverse process's marginal corresponds to no realizable policy's state distribution, and its undo-labels answer questions the forward game never asks. We tested the obvious partial repair: a *spawn-aware* noising walk that injects real forward spawns (probability 0.5 per step) during reverse play, so the training marginal contains the spawn debris that stochastic play produces. It does not help: after 40k iterations the spawn-aware denoiser reaches the 256 tile in 6.2% of games versus random play's 7.1% (mean max-tile 103 vs. 106; greedy-merge 238). Fixing the marginal is not enough, because the labels remain wrong: each undo-label answers "what deterministic slide produced this board," whereas acting well in the real game requires an expectation over spawn randomness. Repairing this demands a fundamentally different reverse process (marginalizing over spawns in the label distribution, i.e. learning $\mathbb{E}_{\text{spawn}}$-aware targets), not a data tweak—evidence that the boundary is structural rather than an artifact of the training set.

6 Discussion: a practitioner's checklist

The strong hypothesis fails; a precise and more useful statement survives. Before reaching for scramble inversion, ask:

1. **Can you run the dynamics backward from the goal, deterministically?** If randomness is exogenous (2048), stop; nothing short of a stochastic-aware reverse process will do.
2. **Is the state space too large to bootstrap a value function across?** If yes (cube, 24-puzzle), scramble inversion is likely the *best* tool, not merely a viable one. If the space is small (8-puzzle, Hanoi), value learning will match it and beat it on optimality.
3. **Are actions irreversible?** Expect deadlock-blindness; budget for the value-veto hybrid, which in our experiments is strictly the best policy (Sokoban: 93.0%).
4. **Are actions continuous?** Keep the distribution: discretize or sample; never regress the mean.
5. **Do your noising walks actually reach the states you must solve?** Measure it (an oracle or proxy distance suffices): on Hanoi, 1200-step walks cover $\leq$12% of the diameter and the learned policy is perfect inside that shell and useless outside it. Schedule coverage, not capacity, sets the ceiling.
6. Conditioning, partial observability, unordered solutions, and non-monotone progress required *no* adaptation in our experiments.

Limitations. Three seeds per configuration (single seed for the 24-puzzle, where the gap is 166×); one hardware platform; MLP backbones only; greedy/1-step solvers at evaluation (stronger search lifts both methods); Sokoban and Peg Solitaire instances from one generator family; 2048 baselines are heuristic rather than learned.

7 Reproducibility

Ten vectorized pure-PyTorch environments, each with a validation gate that reproduces a known exact quantity (state counts, diameters, GF(2) null-space dimension, reverse-step exactness, replay identities).

Machine-readable results in `paper2_data/`; every trainer, evaluator, and figure script at https://github.com/aamirkhani/rubiks-diffusion. Total compute: ∼35 GPU-hours on one RTX 5080 laptop.

References

[1] F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi. Solving the Rubik's cube with deep reinforcement learning and search. *Nature Machine Intelligence*, 1(8):356–363, 2019.

[2] M. Andrychowicz et al. Hindsight experience replay. In *NeurIPS*, 2017.

[3] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg. Structured denoising diffusion models in discrete state-spaces. In *NeurIPS*, 2021.

[4] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.

[5] C. Florensa, D. Held, M. Wulfmeier, M. Zhang, and P. Abbeel. Reverse curriculum generation for reinforcement learning. In *CoRL*, 2017.

[6] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.

[7] M. Janner, Y. Du, J. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. In *ICML*, 2022.

[8] A. Khani. Scramble inversion as discrete denoising diffusion: A matched-compute study on the Rubik's cube group. Preprint, 2026.

[9] C. Resnick, R. Raileanu, S. Kapoor, A. Peysakhovich, K. Cho, and J. Bruna. Backplay: "Man muss immer umkehren". arXiv:1807.06919, 2018.

[10] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *ICML*, 2015.

[11] Z. Sun and Y. Yang. DIFUSCO: Graph-based diffusion solvers for combinatorial optimization. In *NeurIPS*, 2023.

[12] K. Takano. Self-supervision is all you need for solving Rubik's cube. *Transactions on Machine Learning Research*, 2023.

A Hyperparameters

B Validation gates

Each environment must reproduce a known exact quantity before training: 8-puzzle BFS = 181,440 states, diameter 31; Hanoi BFS = 3^{10} states, goal eccentricity $2^{10}-1 = 1023$; Lights Out GF(2) null space of dimension 2; maze BFS field = Manhattan distance on empty grids; Sokoban and Peg Solitaire reverse walks replay to the goal under forward dynamics; pendulum and Mountain Car reverse steps invert forward steps to $<10^{-6}$; 2048 slides conserve tile mass; 24-puzzle scrambles preserve permutation parity by construction.

Domain	S	$\lvert V\rvert$	M	h_1/h_2/blocks	batch	iters	K
24-puzzle	25	25	4	5000/1000/4	10k	62k	300
8-puzzle	9	9	4	1024/512/2	10k	20k	31
maze / POMDP	225	4 / 5	4	2048/1024/3	8192	60k	60
Sokoban	64	7	4	2048/1024/3	8192	80k	40
pendulum	3 feats	—	21 bins	256×3 SiLU	8192	30k	200
2048	16	12	4	2048/1024/3	8192	40k	40
Hanoi	10	3	6	1024/512/2	8192	20k	1200
Lights Out	25	2	25	2048/1024/3	8192	20k	25
Mountain Car	3 feats	—	3	256×3 SiLU	8192	20k	250
Peg Solitaire	49	3	196	2048/1024/3	4096	30k	31

Table 2: Per-domain configuration. Optimizer Adam (10^{-3}, $\times 0.1$ at 80%), bf16 autocast throughout; DAVI target-net sync at loss < 0.05 or staleness cap. Value baselines share every setting.